1. Итак, для выполнения данного задания создаю новую директорию проекта и новый namespace (kube-45), в котором буду работать:

```
anastasia@compute-vm:~/project-kube-45$ kubectl create namespace kube-45
namespace/kube-45 created
anastasia@compute-vm:~/project-kube-45$ kubectl get namespaces
NAME                   STATUS   AGE
default                Active   2d
kube-45                Active   5s
kube-node-lease        Active   2d
kube-public            Active   2d
kube-system            Active   2d
kubernetes-dashboard   Active   2d
```

2. Задаю созданный namespace как основной с помощью команды kubectl config set-context --current --namespace=kube-45
3. Создаю сервисный аккаунт с помощью команды kubectl create serviceaccount kuber и дальше проверяю, что он создался:

```
anastasia@compute-vm:~/project-kube-45$ kubectl create serviceaccount kuber
serviceaccount/kuber created
anastasia@compute-vm:~/project-kube-45$ kubectl get serviceaccounts
NAME      AGE
default   4m37s
kuber     14s
```

4. Создаю роль с привязкой к сервисному аккаунту, которая разрешает читать и записывать конфигурацию подов. Это можно сделать с помощью команд или .yaml файла, где будут описаны все необходимые действия:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: kuber-role
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
```

   – rbac.authorization.k8s.io – группа api, которая управляет авторизацией
   – resources: на какой ресурс выдаем права
   – verbs: какие действия разрешены этой роли (get, list, watch – по сути чтение, просмотр, получение информации; create – создание; update, patch - изменение)
   – Выданы все действия, кроме удаления
5. Применяю конфигурацию kubectl apply –f role.yaml и затем проверяю:

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f role.yaml
role.rbac.authorization.k8s.io/kuber-role created
anastasia@compute-vm:~/project-kube-45$ kubectl get roles
NAME         CREATED AT
kuber-role   2026-09-25T18:11:58Z
```

6. Также создаю файл, который свяжет сервисный аккаунт с ролью rolebinding.yaml, можно конечно командой, но она будет слишком громоздкой и не удобочитаемой:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: binding
subjects:
  - kind: ServiceAccount
    name: kuber
roleRef:
  kind: Role
  name: kuber-role
  apiGroup: rbac.authorization.k8s.io
```

7. Применяю конфигурацию kubectl apply –f rolebinding.yaml

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/binding created
anastasia@compute-vm:~/project-kube-45$ kubectl get rolebindings
NAME      ROLE              AGE
binding   Role/kuber-role   22s
```

8. Создаю манифест пода для проверки созданного сервисного аккаунта и назначенных ему прав:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-app
spec:
  serviceAccountName: kuber
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```

9. Применяю конфигурацию и проверяю:

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f pod-seracc.yaml
pod/nginx-app created
anastasia@compute-vm:~/project-kube-45$ kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
nginx-app   1/1     Running   0          9s
```

10. Еще можно проверить, что поду назначен наш созданный сервисный аккаунт с помощью команды kubectl describe pod nginx-app:

```
anastasia@compute-vm:~/project-kube-45$ kubectl describe pod nginx-app
Name:             nginx-app
Namespace:        kube-45
Priority:         0
Service Account:  kuber
Node:             minikube/192.168.49.2
Start Time:       Fri, 25 Sep 2026 18:38:43 +0000
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.0.26
```

11. Проверим, что назначенные права работают, есть все права кроме прав на удаление. Заходим на под kubectl exec –it nginx-app -- /bin/bash. Когда мы оказались внутри необходимо взять токен сервисного аккаунта, он находится по пути cat /var/run/secrets/kubernetes.io/serviceaccount/tokenи добавляем значение в переменную $TOKEN. Тоже самое делаем и с переменной $NAMESPACE (kube-45), и добавляем дефолтный api сервер APISERVER=https://kubernetes.default.svc
12. Теперь проверим права, например, на чтение, должен быть положительным
    curl --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
     --header "Authorization: Bearer $TOKEN" \
     $APISERVER/api/v1/namespaces/$NAMESPACE/pods
    То есть выдаст нам список JSON где будет наш под

```
root@nginx-app:/# curl --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
     --header "Authorization: Bearer $TOKEN" \
     $APISERVER/api/v1/namespaces/$NAMESPACE/pods
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "41773"
  },
  "items": [
    {
      "metadata": {
        "name": "nginx-app",
        "namespace": "kube-45",
        "uid": "bdb0dba0-74f4-4484-8275-c4bba8aa1a3b",
        "resourceVersion": "38477",
        "generation": 1,
        "creationTimestamp": "2026-09-25T18:52:37Z",
```

13. Теперь проверяем права на удаление, должны отсутствовать
    curl --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
     --header "Authorization: Bearer $TOKEN" \
     --request DELETE \
     $APISERVER/api/v1/namespaces/$NAMESPACE/pods/nginx-app

```
root@nginx-app:/# curl --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt      --header "Authorization: Bearer $TOKEN"      --request DE
LETE      $APISERVER/api/v1/namespaces/$NAMESPACE/pods/nginx-app
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "pods \"nginx-app\" is forbidden: User \"system:serviceaccount:kube-45:kuber\" cannot delete resource \"pods\" in API group \"\" in th
e namespace \"kube-45\"",
  "reason": "Forbidden",
  "details": {
    "name": "nginx-app",
    "kind": "pods"
  },
  "code": 403
}root@nginx-app:/#
```

    Вывод показывает, что он не может удалить, об этом говорит код 403

14. Перехожу к созданию нового сервиса типа ClusterIP, который обслуживает приложение в кластере. Для этого необходимо создать файл service-clusterip.yaml с следующим содержимым:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: clusterip
spec:
  type: ClusterIP
  selector:
    app: nginx-app
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

Важное НО, необходимо установить label в файле с описанием сервисного аккаунта, так как на него будет ссылаться пункт app в файле, где описывается clusteip

```yaml
  metadata:
    name: nginx-app
    labels:
      app: nginx-app
```

15. Применяю конфигурацию и проверяю:

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f service-clusterip.yaml
service/clusterip created
anastasia@compute-vm:~/project-kube-45$ kubectl get services
NAME        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
clusterip   ClusterIP   10.100.203.111  <none>        80/TCP    45s
```

16. Теперь необходимо протестировать, для этого можно создать тестовый под # Запустим временный под для теста kubectl run test-pod --image=busybox --rm -it -- wget -qO- http://clusterip (имя моего порта). Можно еще конечно с помощью port-forward, но так быстрее. Должна выйти привественная страница nginx:

```
anastasia@compute-vm:~/project-kube-45$ kubectl run test-pod --image=busybox --rm -it --
- wget -qO- http://clusterip
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

17. Далее также создаю новый сервис типа NodePort, который экспонирует приложение на определенном порту на каждом рабочем узле кластера. Небольшое уточнение, я уже использовала этот метод в первой домашней работе по куберу, без этого способа я не могла открыть в браузере страницу кубера. Итак, создаю файл service-nodeport.yaml с следующим содержимым:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nodeport
spec:
  type: NodePort
  selector:
    app: nginx-app
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
      protocol: TCP
```

   - type: NodePort — сервис доступен снаружи через порт на узле
   - nodePort: 30080 — конкретный порт на узле

18. Применяю конфигурацию и проверяю:

```
anastasia@compute-vm:~/project-kube-45$ kubectl get service
NAME        TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
clusterip   ClusterIP   10.100.203.111   <none>        80/TCP         17m
nodeport    NodePort    10.97.26.56      <none>        80:30080/TCP   12s
```

19. Теперь проверяем работу NodePort, для этого узнаем ip адрес minikube с помощью команды minikube ip (192.168.49.2):

```
anastasia@compute-vm:~/project-kube-45$ curl http://192.168.49.2:30080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

20. Создаю новый сервис типа LoadBalancer, который автоматически создает внешний балансировщик нагрузки для доступа к приложению из внешней сети

```yaml
apiVersion: v1
kind: Service
metadata:
  name: loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: nginx-app
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

   – type: LoadBalancer — кубер запросит у облачного провайдера внешний балансировщик

21. Применяем конфигурацию и проверяем:

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f service-loadbalancer.yaml
service/loadbalancer created
anastasia@compute-vm:~/project-kube-45$ kubectl get services
NAME           TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
clusterip      ClusterIP      10.100.203.111   <none>        80/TCP         27m
loadbalancer   LoadBalancer   10.97.71.127     <pending>     80:31082/TCP   10s
nodeport       NodePort       10.97.26.56      <none>        80:30080/TCP   9m31s
```

22. Теперь нужно открыть туннель миникуба с помощью команды minikube tunnel. Также нужно взять айпи созданного нами сервиса и curl:

```
anastasia@compute-vm:~/project-kube-45$ minikube tunnel
Status:
        machine: minikube
        pid: 17661
        route: 10.96.0.0/12 -> 192.168.49.2
        minikube: Running
        services: [loadbalancer]
    errors:
                minikube: no errors
                router: no errors
                loadbalancer emulator: no errors
```

```
anastasia@compute-vm:~/project-kube-45$ curl http://10.97.71.127:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

23. Для создания Persistent Volume мне необходимо зайти на minikube (с помощью minikube ssh) и там проверяю наличие пути /mnt/data для создания volume:

```
docker@minikube:~$ sudo mkdir -p /mnt/data
docker@minikube:~$ sudo chmod 777 /mnt/data
```

24. Создаю PV с использованием локального хранилища. В файле persistent-volume.yaml прописываю следующее:

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pers-vol
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: local-storage
  persistentVolumeReclaimPolicy: Retain
  local:
    path: /mnt/data
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - minikube
```

- Задаем размер хранилища 1Gi
- accessModes: - ReadWriteOnce определяет как можно использовать этот том. То есть Чтение и запись разрешены, но только с ОДНОЙ ноды одновременно.
- storageClassName: по сути задает тег по которому его можно найти
- PVRP: при удалении сохраняет данные
- nodeAffinity — привязка к конкретному узлу (в нашем случае minikube)

25. Далее создаю Persistent Volime Claim, чтобы запросить доступ к Persistent Volume, который мы создали в предыдущем шаге. Для этого необходимо создать файл persistent-vol-claim.yaml со следующим содержимым:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: persistent-vol-claim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local-storage
  resources:
    requests:
      storage: 1Gi
```

- accessModes — должен совпадать с PV
- storageClassName — должен совпадать с PV, иначе PVC не найдёт PV

26. Применяю конфигурацию и проверяю. Статус Bound означает, что они связаны:

```
anastasia@compute-vm:~/project-kube-45$ kubectl get pv
NAME       CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                       STORAGECLASS    VOLUMEATTRIBUTESCLASS   REASON   AGE
pers-vol   1Gi        RWO            Retain           Bound    kube-45/persistent-vol-claim   local-storage   <unset>                          9m54s
anastasia@compute-vm:~/project-kube-45$ kubectl get pvc
NAME                 STATUS   VOLUME     CAPACITY   ACCESS MODES   STORAGECLASS    VOLUMEATTRIBUTESCLASS   AGE
persistent-vol-claim   Bound    pers-vol   1Gi        RWO            local-storage   <unset>                 96s
```

27. Создаю Pod, который монтирует PVC в качестве тома и выполняет операции чтения и записи данных. Для этого создаю файл pod-with-pvc.yaml со следующим содержимым:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod
spec:
  containers:
    - name: storage-container
      image: busybox:latest
      command: ["sh", "-c", "while true; do echo 'Hello from Pod!' >> /data/hello.txt; sleep 5; done"]
      volumeMounts:
        - name: volume-app
          mountPath: /data
  volumes:
    - name: volume-app
      persistentVolumeClaim:
        claimName: persistent-vol-claim
```

- command — под будет каждые 5 секунд записывать «Hello from Pod!» в файл /data/hello.txt
- volumeMounts — монтирует том в контейнер по пути /data
- volumes — объявляет том, источником которого является PVC persistent-vol-claim

28. Применяю конфигурацию и проверяю, для этого захожу в созданный под и проверяю файл, который должен был создастся, вместе с содержимым:

```
anastasia@compute-vm:~/project-kube-45$ kubectl exec -it storage-pod -- sh
/ # cat /data/hello.txt
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
```

29. Далее проверяем на узле minikube по тому же пути:

```
anastasia@compute-vm:~/project-kube-45$ minikube ssh
Linux minikube 6.8.0-142-generic #142-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
docker@minikube:~$ cat /mnt/data/hello.txt
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
Hello from Pod!
```

30. Итак, создаю StorageClass, которое будет имитировать облачное хранилище. Для этого создаю файл storage-class-cloud.yaml:

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cloud-storage
provisioner: k8s.io/minikube-host-path
parameters:
  type: gp2
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

- provisioner — кто создаёт хранилище. В облаке это плагин провайдера. Это будет имитация
- parameters — параметры для провайдера (тип диска и прочее)
- reclaimPolicy: Delete — при удалении PVC данные удаляются
- volumeBindingMode: Immediate — привязывать PV сразу при создании Persistent Volume Claim

31. Применяем и проверяем:

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f storage-class-cloud.yaml
storageclass.storage.k8s.io/cloud-storage created
anastasia@compute-vm:~/project-kube-45$ kubectl get storageclass
NAME                 PROVISIONER                RECLAIMPOLICY   VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
cloud-storage        k8s.io/minikube-host-path  Delete          Immediate           false                  9s
```

32. Теперь необходимо создать новое PVC так как старое нельзя изменить после создания. Создаю файл pers-vol-claim-cloud.yaml:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-cloud
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: cloud-storage
  resources:
    requests:
      storage: 1Gi
```

33. Применяю и проверяю:

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f pers-vol-claim-cloud.yaml
persistentvolumeclaim/pvc-cloud created
anastasia@compute-vm:~/project-kube-45$ kubectl get pvc
NAME                 STATUS    VOLUME     CAPACITY   ACCESS MODES   STORAGECLASS    VOLUMEATTRIBUTESCLASS   AGE
persistent-vol-claim   Bound     pers-vol   1Gi        RWO            local-storage   <unset>                 24m
pvc-cloud            Pending                                        cloud-storage   <unset>                 20s
```

34. Создаю новый под с новым PVC. Необходимо создать файл с названием pod-with-pvc-cloud.yaml со следующим содержимым:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod-cloud
spec:
  containers:
    - name: storage-container
      image: busybox:latest
      command: ["sh", "-c", "while true; do echo 'Hello from Cloud Storage!' >> /data/hello.txt; sleep 5; done"]
      volumeMounts:
        - name: cloud-volume
          mountPath: /data
  volumes:
    - name: cloud-volume
      persistentVolumeClaim:
        claimName: pvc-cloud
```

35. Применяю и проверяю:

```
anastasia@compute-vm:~/project-kube-45$ kubectl apply -f pod-with-pvc-cloud.yaml
pod/storage-pod-cloud created
anastasia@compute-vm:~/project-kube-45$ kubectl get pods
NAME              READY   STATUS    RESTARTS   AGE
nginx-app         1/1     Running   0          168m
storage-pod       1/1     Running   0          19m
storage-pod-cloud   0/1     Pending   0          47s
```